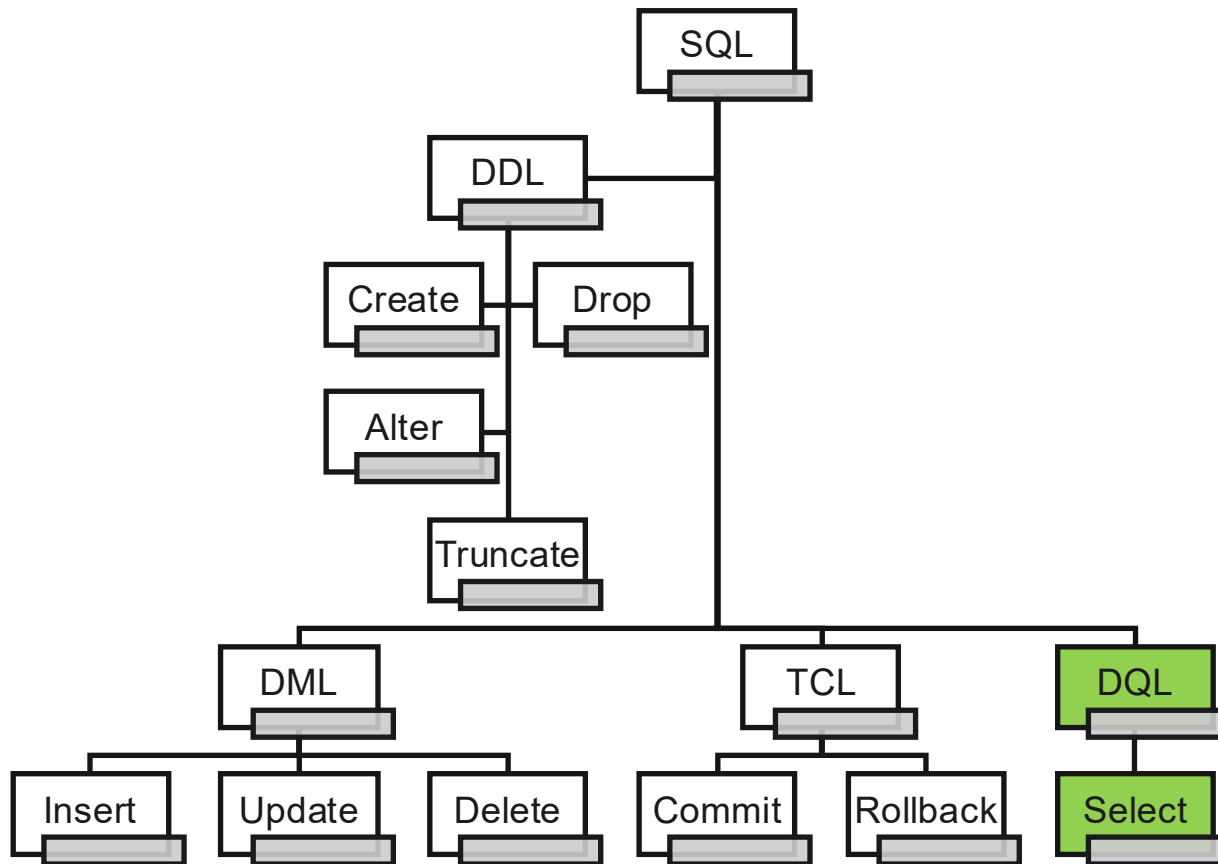


SQL: DQL

Vanessa Braganholo

vanessa@ic.uff.br

Aula de Hoje



Esquema usado nos exemplos

```
CREATE TABLE DEPARTAMENTO (
  COD_DEPTO INT NOT NULL,
  NOME VARCHAR(15) NOT NULL,
  CPF_GERENTE VARCHAR(11),
  DATA_INICIO_GERENTE DATE,
  PRIMARY KEY (COD_DEPTO),
  UNIQUE (NOME),
  FOREIGN KEY (CPF_GERENTE) REFERENCES EMPREGADO(CPF));
```

```
CREATE TABLE EMPREGADO (
  NOME VARCHAR(15) NOT NULL,
  INICIALM CHAR,
  SOBRENOME VARCHAR(15) NOT NULL,
  CPF VARCHAR(11) NOT NULL,
  DNASCIMENTO DATE,
  ENDERECO VARCHAR(30),
  SEXO CHAR,
  SALARIO FLOAT,
  CPF_SUPERVISOR VARCHAR(11),
  COD_DEPTO INT NOT NULL,
  PRIMARY KEY (CPF),
  FOREIGN KEY (CPF_SUPERVISOR) REFERENCES EMPREGADO (CPF),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE DEPTO_LOCALIZACOES (
  COD_DEPTO INT NOT NULL,
  LOCAL VARCHAR(15) NOT NULL,
  PRIMARY KEY (COD_DEPTO),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE PROJETO (
  COD_PROJ INT NOT NULL,
  NOME VARCHAR(30) NOT NULL,
  LOCAL VARCHAR(15),
  COD_DEPTO INT NOT NULL,
  PRIMARY KEY (COD_PROJ),
  UNIQUE (NOME),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE TRABALHA_EM (
  CPF VARCHAR(11) NOT NULL,
  COD_PROJ INT NOT NULL,
  HORAS FLOAT NOT NULL,
  PRIMARY KEY (CPF, COD_PROJ),
  FOREIGN KEY (CPF) REFERENCES EMPREGADO (CPF),
  FOREIGN KEY (COD_PROJ) REFERENCES PROJETO (COD_PROJ));
```

```
CREATE TABLE DEPENDENTE (
  EMP_CPF VARCHAR(11) NOT NULL,
  NOME VARCHAR(15) NOT NULL,
  SOBRENOME VARCHAR(15) NOT NULL,
  SEXO CHAR,
  DNASCIMENTO DATE,
  PARENTESCO VARCHAR(8),
  PRIMARY KEY (EMP_CPF, NOME),
  FOREIGN KEY (EMP_CPF) REFERENCES EMPREGADO(CPF));
```

EMPREGADO

Í	NOME	INICIA...	SOBR...	CPF	DNAS...	ENDE...	SEXO	SALA...	CPF_S...	COD_DEPTO
	John	B	Smith	12345...	1965-01...	731 Fon...	M	30000	33344...	5
	Franklin	T	Wong	33344...	1955-12...	638 Vos...	M	40000	88866...	5
	Joyce	A	English	45345...	1972-07...	5631 Ri...	F	25000	33344...	5
	Ramesh	K	Narayan	66688...	1962-09...	975 Fire...	M	38000	33344...	5
	James	E	Borg	88866...	1937-11...	450 Sto...	M	55000	NULL	1
	Jennifer	S	Wallace	98765...	1941-06...	291 Ber...	F	43000	88866...	4
	Ahmad	V	Jabbar	98798...	1969-03...	980 Dall...	M	25000	98765...	4
	Alicia	J	Zelaya	99988...	1968-01...	3321 C...	F	25000	98765...	4

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

DEPTO_LOCALIZACOES

COD_DEPTO	LOCAL
1	Houston
4	Staffor
5	Bellaire

PROJETO

COD_PROJ	NOME	LOCAL	COD_DEPTO
1	ProdutoX	Bellaire	5
2	ProdutoY	Sugarland	5
3	ProdutoZ	Houston	5
10	Automatizacao	Stafford	4
20	Reorganizacao	Houston	1
30	Novos Beneficios	Stafford	4

TRABALHA_EM

CPF	COD_PROJ	HORAS
12345678900	1	32.5
12345678900	2	7.5
33344555500	2	10
33344555500	3	10
33344555500	10	10
33344555500	20	10
45345345300	1	20
45345345300	2	20
66688444400	3	40
88866555500	20	40
98765432100	20	15
98765432100	30	20
98798798700	10	35
98798798700	30	5
99988777700	10	10
99988777700	30	30

DEPENDENTE

EMP_CPF	NOME	SOBRENOME	SEXO	DNASCIMENTO	PARENTESCO
12345678900	Alice	Smith	F	1988-12-30	FILHA
12345678900	Elizabeth	Smith	F	1967-05-05	CONJUGE
12345678900	Michael	Smith	M	1988-01-04	FILHO
33344555500	Alice	Braga	F	1986-04-05	FILHA
33344555500	Joy	Silva	F	1958-05-03	CONJUGE
33344555500	Theodore	Silva	M	1983-10-25	FILHO
98765432100	Abner	Soares	M	1942-02-28	CONJUGE

Consultas mais “Complexas”

Recursos adicionais que permitem que os usuários especifiquem consultas mais complexas:

- Tabelas de junção
- Junções externas
- Operações de conjuntos

Tabelas de Junção

Tabela de Junção

- Permite aos usuários especificar uma tabela resultante de uma operação de junção na cláusula FROM de uma consulta

A cláusula FROM na consulta a seguir contém uma única tabela de junção

```
SELECT e.NOME, SOBRENOME, ENDERECO
FROM (EMPREGADO AS e JOIN DEPARTAMENTO AS d
      ON e.COD_DEPTO = d.COD_DEPTO)
WHERE d.NOME = 'Pesquisa';
```

Consulta com 2 Junções

```
SELECT COD_PROJ, SOBRENOME, ENDERECO, DNASCIMENTO
FROM PROJETO p JOIN DEPARTAMENTO d
      ON p.COD_DEPTO = d.COD_DEPTO
      JOIN EMPREGADO e ON e.CPF = d.CPF_GERENTE
WHERE p.LOCAL = 'Niterói';
```

Junções Externas

Tipos de Junção

INNER JOIN

LEFT OUTER JOIN

RIGHT OUTER JOIN

FULL OUTER JOIN

INNER JOIN

O tipo padrão de junção em uma tabela de junção

Tupla é incluída no resultado somente se uma tupla combinar na outra relação

Exemplo de INNER Join

```
SELECT e.NOME, SOBRENOME, ENDERECO
FROM (EMPREGADO AS e JOIN DEPARTAMENTO AS d
        ON e.COD_DEPTO = d.COD_DEPTO)
WHERE d.NOME = 'Pesquisa';
```

NOME	SOBRENOME	ENDERECO
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
John	Smith	731 Fondren, Houston, TX
Joyce	English	5631 Rice, Houston, TX

Relembrando o conteúdo das tabelas

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

EMPREGADO

NOME	INICIA...	SOBR...	CPF	DNAS...	ENDE...	SEXO	SALA...	CPF_S...	COD_DEPTO
John	B	Smith	12345...	1965-01...	731 Fon...	M	30000	33344...	5
Franklin	T	Wong	33344...	1955-12...	638 Vos...	M	40000	88866...	5
Joyce	A	English	45345...	1972-07...	5631 Ri...	F	25000	33344...	5
Ramesh	K	Narayan	66688...	1962-09...	975 Fire...	M	38000	33344...	5
James	E	Borg	88866...	1937-11...	450 Sto...	M	55000	NULL	1
Jennifer	S	Wallace	98765...	1941-06...	291 Ber...	F	43000	88866...	4
Ahmad	V	Jabbar	98798...	1969-03...	980 Dall...	M	25000	98765...	4
Alicia	J	Zelaya	99988...	1968-01...	3321 C...	F	25000	98765...	4

LEFT OUTER JOIN

Toda tupla na tabela da esquerda precisa aparecer no resultado;

Se não existir uma tupla combinando na tabela da direita, tupla no resultado é preenchida com valores NULL para os atributos da tabela da direita

Exemplo de LEFT OUTER JOIN

Selecionar os empregados e seus supervisores. Empregados que não possuem supervisor devem mesmo assim aparecer na resposta

```
SELECT e.NOME AS NOME_EMPREGADO, s.NOME AS NOME_SUPERVISOR  
FROM EMPREGADO e LEFT OUTER JOIN EMPREGADO s  
      ON e.CPF_SUPERVISOR = s.CPF;
```

Exemplo de LEFT OUTER JOIN

```

SELECT e.NOME AS NOME_EMPREGADO, s.NOME AS NOME_SUPERVISOR
FROM EMPREGADO e LEFT OUTER JOIN EMPREGADO s
           ON e.CPF_SUPERVISOR = s.CPF;
  
```

NOME_EMPREGADO	NOME_SUPERVISOR
James	NULL
Franklin	James
Ramesh	Franklin
John	Franklin
Jennifer	James
Ahmad	Jennifer
Alicia	Jennifer
Joyce	Franklin



RIGHT OUTER JOIN

Toda tupla na tabela da direita precisa aparecer no resultado

Se não existir uma tupla combinando na tabela da esquerda, tupla no resultado é preenchida com valores NULL para os atributos da tabela da esquerda

OBS: RIGHT OUTER JOIN só existe no SQLite a partir da versão 3.39 (junho de 2022)

- Confira a versão do seu SQLite com `SELECT sqlite_version();`
- Em versões antigas, é possível para simular com LEFT JOIN invertido

Exemplo de RIGHT OUTER JOIN

Selecionar os departamentos e os projetos de cada um.
Departamentos que não possuem projetos devem mesmo assim aparecer na resposta

```
SELECT p.NOME AS PROJETO, d.NOME AS DEPARTAMENTO  
FROM PROJETO p RIGHT OUTER JOIN DEPARTAMENTO d  
    ON p.COD_DEPTO = d.COD_DEPTO;
```

Relembrando o conteúdo das tabelas

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

Não há departamento sem projeto, portanto essa consulta retorna o mesmo que o INNER JOIN retornaria.

Se adicionarmos um departamento novo, sem projeto, a resposta seria conforme o slide seguinte.

PROJETO

COD_PROJ	NOME	LOCAL	COD_DEPTO
1	ProdutoX	Bellaire	5
2	ProdutoY	Sugarland	5
3	ProdutoZ	Houston	5
10	Automatizacao	Stafford	4
20	Reorganizacao	Houston	1
30	Novos Beneficios	Stafford	4

Exemplo de RIGHT OUTER JOIN

```

SELECT p.NOME AS PROJETO, d.NOME AS DEPARTAMENTO
FROM PROJETO p RIGHT OUTER JOIN DEPARTAMENTO d
ON p.COD_DEPTO = d.COD_DEPTO;
  
```

PROJETO	DEPARTAMENTO
ProdutoX	Pesquisa
ProdutoY	Pesquisa
ProdutoZ	Pesquisa
Automatizacao	Administracao
Reorganizacao	Sede administrativa
Novos Beneficios	Administracao
NULL	Novo



FULL OUTER JOIN

Toda tupla das duas tabelas precisa aparecer no resultado

Se não existir tupla combinando na tabela da direita ou da esquerda, tupla no resultado é preenchida com NULL no lado faltante

OBS: FULL OUTER JOIN só existe no SQLite a partir da versão 3.39 (junho de 2022)

- Confira a versão do seu SQLite com `SELECT sqlite_version();`
- Em versões antigas, é possível para simular com UNION

Exemplo de FULL OUTER JOIN

Selecionar os departamentos e seus gerentes. Departamentos que ainda não têm gerente e empregados que não gerenciam nenhum departamento devem aparecer na resposta

```
SELECT d.NOME AS DEPARTAMENTO, e.NOME AS GERENTE  
FROM DEPARTAMENTO d FULL OUTER JOIN EMPREGADO e  
    ON d.CPF_GERENTE = e.CPF;
```

Exemplo de FULL OUTER JOIN

```
SELECT d.NOME AS DEPARTAMENTO, e.NOME AS GERENTE
FROM DEPARTAMENTO d FULL OUTER JOIN EMPREGADO e
ON d.CPF_GERENTE = e.CPF;
```

DEPARTAMENTO	GERENTE
Sede administrativa	James
Administracao	Jennifer
Pesquisa	Franklin
Novo	NULL
NULL	Ramesh
NULL	John
NULL	Ahmad
NULL	Alicia
NULL	Joyce



Operações de Conjuntos

Recapitulando: Tabelas como Conjuntos

SQL não elimina automaticamente tuplas duplicadas nos resultados das consultas (resultado não é um conjunto)

Para garantir que o resultado contenha apenas tuplas distintas, usa-se a palavra-chave **DISTINCT** na cláusula **SELECT**

Recapitulando: Tabelas como Conjuntos

```
SELECT SALARIO  
FROM EMPREGADO;
```

SALARIO
55000
40000
38000
30000
43000
25000
25000
25000

```
SELECT DISTINCT SALARIO  
FROM EMPREGADO;
```

SALARIO
55000
40000
38000
30000
43000
25000

Tabelas como Conjuntos

Operações de conjunto (**duplicatas são eliminadas**):

- UNION
- EXCEPT (diferença)
- INTERSECT

Operações multi-conjuntos correspondentes (**duplicatas NÃO são eliminadas**):

- UNION ALL
- EXCEPT ALL (não existe no SQLite)
- INTERSECT ALL (não existe no SQLite)

Tabelas como Conjuntos

Em todos os casos, tabelas precisam ser **compatíveis para união**

- Colunas nas mesmas posições nas duas tabelas unidas precisam ter domínios compatíveis

UNION

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace, seja como gerente do departamento onde o projeto é realizado ou como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME  
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e  
WHERE p.COD_DEPTO = d.COD_DEPTO  
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION

```
SELECT p.COD_PROJ, p.Nome  
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e  
WHERE p.COD_PROJ = t.COD_PROJ  
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

UNION

Domínio das
colunas é o
mesmo

Faça uma lista de todos os nomes de projetos e nomes de departamentos

```
SELECT p.NOME  
FROM PROJETO p  
UNION  
SELECT d.NOME  
FROM DEPARTAMENTO d
```

NOME
Administracao
Automatizacao
Novos Beneficios
Pesquisa
ProdutoX
ProdutoY
ProdutoZ
Reorganizacao
Sede administrativa

UNION

Domínio das
colunas é o
mesmo, mas
nome é diferente

Faça uma lista de todos os sobrenomes de empregados e nomes de departamentos

```
SELECT e.SOBRENOME
FROM EMPREGADO e
UNION
SELECT d.NOME
FROM DEPARTAMENTO d
```

SOBRENOME
Administracao
Borg
English
Jabbar
Narayan
Pesquisa
Sede administrativa
Smith
Wallace
Wong
Zelaya

UNION

Mesmos
nomes de
colunas

Faça uma lista de todos os sobrenomes de empregados e nomes de departamentos

```
SELECT e.SOBRENOME AS NOME
FROM EMPREGADO e
UNION
SELECT d.NOME
FROM DEPARTAMENTO d
```

⋮ NOME

Administracao

Borg

English

Jabbar

Narayan

Pesquisa

Sede administrativa

Smith

Wallace

Wong

Zelaya

UNION

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

U

Wallace trabalha
nos projetos 20 e
30.

COD_PROJ	NOME
10	Automatizacao
20	Reorganizacao
30	Novos Beneficios

UNION ALL

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION ALL

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

UNION ALL

Wallace trabalha
nos projetos 20 e
30.

COD_PROJ	NOME
10	Automatizacao
30	Novos Beneficios
20	Reorganizacao
30	Novos Beneficios

INTERSECT

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace tanto como gerente do departamento onde o projeto é realizado quanto como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

∩

Wallace trabalha
nos projetos 20 e
30.

⋮ COD_PROJ	NOME
30	Novos Benefícios

EXCEPT

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace como gerente do departamento onde o projeto é realizado, mas não como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

-

Wallace trabalha
nos projetos 20 e
30.

COD_PROJ	NOME
10	Automatizacao

Para praticar...

Vamos usar o SQLite

No Google Classroom, baixe o arquivo airport.db e salve numa pasta do seu computador

No VSCode, abra essa pasta pelo menu File / Open Folder

Clique no SQLite Explorer e depois em New Query

Escreva a consulta, clique com o botão da direita e escolha Run Query

O passo a passo completo está no material **SQLite no VSCode**

Atenção: no SQLite a comparação com = diferencia maiúsculas de minúsculas ('ES' não é igual a 'es')

SQLite Online

Se não conseguir instalar a extensão no VSCode, você também pode usar o <https://sqliteonline.com/>



```

aircrafttype (AircraftTypeID, AircraftName)
airport (AirportID, AirportCode, AirportName, CityName,
          CountryCode, NumRunways, NumTerminals)
aircraft (AircraftID, AircraftTypeID, RegNum, LastMaintEnd,
          NextMaintBegin, NextMaintEnd);
          AircraftTypeID referencia aircrafttype
route (RouteID, Origin, Destination, Distance, Duration, RouteStatus)
          Origin references airport (AirportID)
          Destination references airport (AirportID)
class (ClassID, ClassName)
flight (FlightID, RouteID, AircraftID)
          AircraftID referencia aircraft
          RouteID referencia route
flightclass (FlightID, ClassID, MaxSeats, BasePrice)
          FlightID referencia flight
          ClassID referencia class
flightdep (FlightID, DepDay, DepTime); //horários de vôos
          FlightID referencia flight
log (RecordID, ByUser, Note, EventTime)
pax (RecordID, FlightID, FlightDate, ClassID, PaxName, PaxRef, Note)
          FlightID referencia flight
          ClassID referencia class
stats (FlightID, FlightDate, ClassID, CurrSeats, CurrPrice)
          FlightID referencia flight
          ClassID referencia class

```

Exercícios

1. Faça uma consulta que retorna as rotas (RouteID, Origin, Destination) que possuem como origem o aeroporto LHR ou o aeroporto AMS. Use o comando de JOIN apropriado.
2. Repita a consulta anterior, agora usando UNION para produzir a resposta.
3. Faça uma consulta que retorna o código dos aeroportos de origem e destino de cada rota – a consulta deve retornar o código da rota, o código do aeroporto origem (renomeado para Origem) e o código do aeroporto destino (coluna AirportCode) renomeado como Destino. O retorno deve estar ordenado pelo código do aeroporto de origem e logo após, pelo código do aeroporto de destino. Use o comando de JOIN apropriado.
4. Faça uma consulta que retorne todas as informações de voo (flight) e os horários de partida desse voo (tabela flightdep). Use o operador de junção apropriado. O resultado deve estar ordenado por FlightID.
5. Faça uma consulta que retorna os voos (FlightID) e os IDs das classes que esse voo opera. Caso um voo não tenha registro de classe, mesmo assim ele deve aparecer no resultado.

Exercícios

6. Faça uma consulta que retorna os dados dos vôos para os quais existe alguma reserva (tabela pax). Use o comando JOIN.
7. Faça uma consulta que retorna o número do vôo e o nome do passageiro que reservou o vôo (tabela pax). O número do vôo deve aparecer no resultado mesmo se não houver nenhuma reserva para ele. A resposta deve estar ordenada pelo número do vôo e pelo nome do passageiro.
8. Faça uma consulta que retorne o FlightID dos vôos para os quais não existe reserva (use EXCEPT).
9. Repita a consulta anterior, agora retornando, além do FlightID, o AirportCode de Origem e Destino dos vôos

Referência

Sistemas de Banco de Dados. Ramez E. Elmasri, e Shamkant Navathe, 2005, 4a edição, Ed. Pearson - Addison Wesley.

Referência

Tutorial SQLite:

<https://www.sqlitetutorial.net/>

Sintaxe: <https://www.sqlite.org/lang.html>



Referência

W3 Schools (apresenta sintaxe para vários BDs diferentes)

- <https://www.w3schools.com/sql/default.asp>



Agradecimento

Slides baseados no material de Daniel Oliveira

Perguntas?

Vanessa Braganholo

vanessa@ic.uff.br